

PLaND Path to Least Non Determinism

Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo

Affiliations: Both authors are Independent Researchers, San Francisco, CA, USA. Corresponding author: Maddipatla Naga Venkata Sai Krishna; mnvsk97@gmail.com.

Abstract

Language-model workflows often mix ambiguous decisions with routine decisions that explicit rules could handle. We present Path to Least Non Determinism (PLaND), a methodology for moving suitable work from natural-language instructions into executable code. A workflow starts with an English standard operating procedure (SOP). An evolver skill guides a host agent to propose changes using development examples and execution records. Each candidate is compared with the baseline under fixed quality and token-use criteria. We evaluate fixed natural-language and hybrid SOPs on three classification tasks: LEDGAR legal clauses, CFPB consumer complaints, and SpamAssassin email. The reported evaluations use 1,000 LEDGAR clauses, 100 CFPB complaints, and 100 SpamAssassin emails. On LEDGAR, accuracy changed from 93.5% to 92.7%, within the study's two-percentage-point tolerance after accounting for sampling uncertainty. Model calls fell from 1,000 to 589 and token use fell 40.02%. CFPB and SpamAssassin did not meet the acceptance criteria. Three additional paired runs per dataset reproduced these acceptance decisions. These results show that deterministic routing can substantially reduce model use, but the benefit depends on both the task and the allowed accuracy loss. The experiments measure the execution of fixed SOPs; they do not measure how reliably an agent can discover useful rules from run histories without researcher involvement.

Keywords: agentic workflows, agent skills, deterministic execution, language-model agents, workflow optimization, token efficiency, hybrid systems

1 Introduction

Business processes often contain both stable and ambiguous decisions. A language model can interpret unfamiliar inputs and handle exceptions, but it may also spend tokens repeatedly applying the same recognizable rule. For example, a contract clause that explicitly specifies its governing law may be easier to classify than a clause that combines several legal functions. The practical question is which decisions can be handled by code while keeping the workflow's quality within an acceptable range.

Existing infrastructure provides ways to package and execute these workflows. The Agent Skills specification organizes reusable instructions in a `SKILL.md` file with optional supporting files [1]. DeepAgents supports loading skills [2], while LangGraph provides graph-based workflow orchestration [3]. These are existing systems on which a methodology can build, not components introduced or independently benchmarked in this paper.

Related research optimizes model programs, prompts, workflows, and reusable skills. DSPy optimizes language-model pipelines against task metrics [4]. AutoFlow and AFlow generate and improve workflows [5, 6], while Automated Design of Agentic Systems searches over agent designs [7]. SkillOpt, SkillRevise, SkillReducer, and ACES study the optimization or evaluation of reusable skills [8-11]. PLaND focuses on a specific change within a workflow: replacing suitable model-mediated decisions with explicit computation, while retaining model reasoning for the remaining inputs and evaluating the complete result.

PLaND begins with an English standard operating procedure (SOP), then permits a hybrid SOP that combines model instructions with executable rules. Further changes may produce a workflow whose stable steps are explicit graph nodes. This last stage is a proposed direction, not an outcome demonstrated here. In this paper, reducing non-determinism means reducing

dependence on model-mediated execution. It does not mean finding a globally optimal workflow or proving that repeated model outputs become less variable.

We evaluate this approach on LEDGAR, CFPB, and SpamAssassin. These tasks let us test whether code can avoid model calls, whether accuracy remains within a stated tolerance, and whether the same acceptance decision holds across repeated runs. They are individual classification tasks, not complete business processes.

2 Material and Methods

2.1 Skill structure and executable steps

A skill is a directory whose main instructions are written in `SKILL.md`. Optional `references/` files provide supporting instructions or domain material, `scripts/` contains executable code, and `assets/` holds resources such as templates [1]. A reference file in this directory is supporting material for the agent; it is not a bibliographic citation and does not make a decision deterministic by itself.

PLaND distinguishes instructions that a model interprets from code that the runtime executes. We use the term executable step for a Python or Bash operation with defined inputs and outputs. Earlier SOPs call this a command. An executable step contributes to deterministic execution only if the runtime actually calls the code and uses its result. Merely mentioning code in a prompt does not establish that it ran.

Figure 1 illustrates the progression using LEDGAR. The English SOP asks the model to read a clause, identify its legal function, compare the allowed labels, and choose one. The hybrid SOP adds a rule-based classifier. Inputs that the rules cannot resolve return to the model. Section 3.2 explains how the benchmark executes this choice.

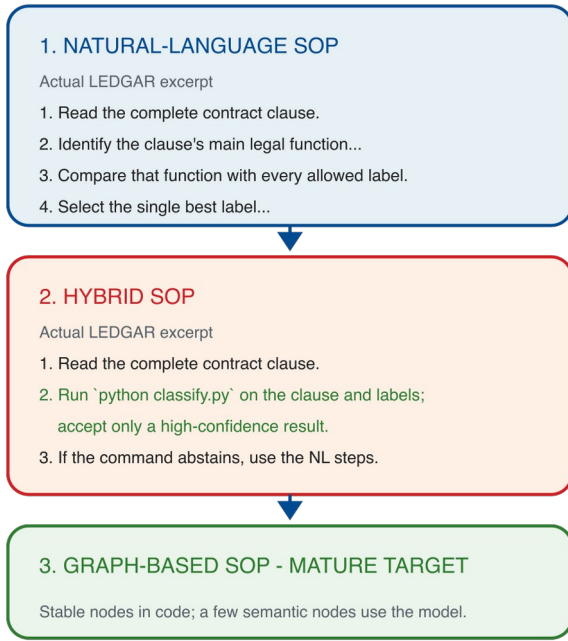


Figure 1. From English instructions to selective code execution. The first two stages summarize the evaluated LEDGAR SOPs. The third is a proposed future organization of a workflow, not a measured result. A graph implementation need not use a particular orchestration framework.

2.2 Evolution and the fixed evaluation boundary

PLaND uses two skills. `generate-initial-version` turns task requirements, approved data sources, and evaluation examples into an initial agent and English SOP. `pland-evolver` guides a host reasoning agent to inspect development runs and propose a bounded change to that SOP and its supporting files. The methodology is task-agnostic: a task-specific runner executes the workflow, and a task-specific scorer checks the output.

The dataset supplies inputs and expected answers. Running a workflow on development inputs produces execution records, including its output, errors, routing decisions, model calls, and tokens. These records can guide candidate revision. Validation cases serve a different purpose: they decide whether the resulting candidate is acceptable. Reserved test cases provide a final assessment after that decision. Thus, datasets and run histories are complementary; the histories are produced by running the workflow on examples from the dataset.

Only the SOP package and its directly related files may change during a paired comparison. The task, model, system prompt, data snapshot, scorer, runtime settings, permissions, and acceptance criteria remain fixed. The comparison tools check case identifiers and recorded fingerprints for mismatches. Figure 2 shows this boundary. The evolver policy permits a bounded search, with a default limit of ten candidates; this policy limit is not a claim that ten candidates were generated in every reported experiment.

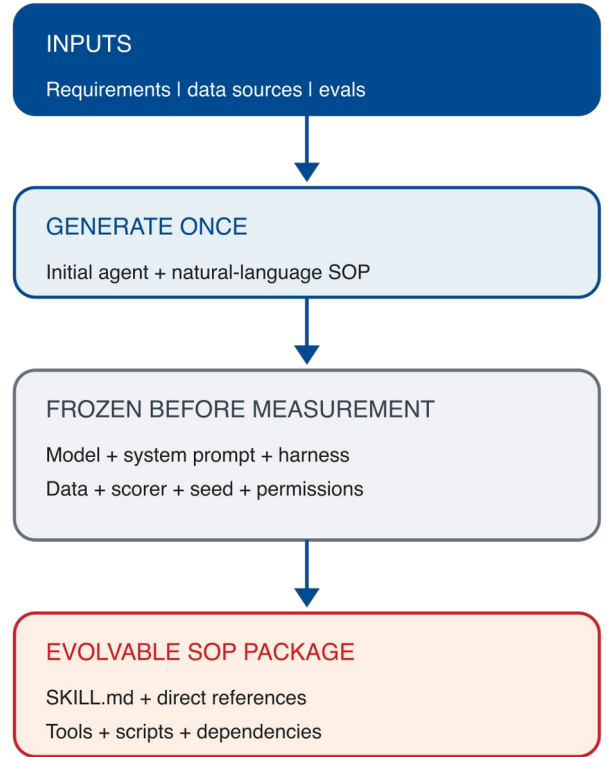


Figure 2. What can change during a comparison. Candidate changes are confined to the SOP package. The same evaluation conditions apply to the baseline and candidate. Development records can guide revisions, but validation and test answers are not supplied as agent inputs.

The experiments below evaluate saved, fixed SOPs and scripts. They establish how those packages behave when executed. They do not separately measure the success rate of the earlier generation step across independent agent attempts. An evaluation of automatic rule discovery would need to record the development histories supplied to the agent, the code it generates, and its performance on new cases. That question is distinct from whether a fixed rule-based route saves model calls.

2.3 Quality and expense

Let W denote the baseline workflow and W' a proposed replacement. $Q(W)$ is its task quality, measured here as classification accuracy. $E(W)$ is its expense, measured here as total model input and output tokens. Q_{min} is the minimum acceptable accuracy. We call a workflow usable under the study criteria when it reaches Q_{min} ; this is the meaning of baseline viability. The symbol ϵ denotes the largest allowed decrease in accuracy relative to the baseline.

The intended comparison is:

$$\begin{aligned} E(W') &< E(W) \\ Q(W) &\geq Q_{min}, \quad Q(W') \geq Q_{min} \\ Q(W') - Q(W) &\geq -\epsilon \end{aligned}$$

For these experiments, Q_{min} is 0.80 and ϵ is 0.02. In plain terms, both workflows must classify at least 80% of cases correctly, and the candidate may lose at most two percentage points of accuracy. A change from 93.5% to

91.5% is a two-point decrease; it is not a 2% relative decrease. Section 3.3 gives the full decision rule, including uncertainty and the minimum token reduction.

These values are study-specific engineering choices recorded in the comparison configuration [12]. They are not thresholds established by the dataset creators or universal standards for these tasks. The records do not provide an application-specific error-cost analysis that would justify 80% or two points for deployment. We therefore interpret acceptance only under these stated tolerances. A real application should choose its accuracy requirements before evaluation, based on the consequences of its errors. Non-inferiority testing

Table 1. Evaluation datasets

Dataset and classification task	Labels	Evaluated examples
LEDGAR legal clause type	10	1,000
CFPB complaint product	10	100
SpamAssassin spam or legitimate email	2	100

LEDGAR uses ten clause labels: Governing Laws, Counterparts, Notices, Entire Agreements, Severability, Amendments, Survival, Assignments, Expenses, and Terms. Selection preserves the source dataset's training, validation, and test boundaries. CFPB uses ten product categories from a saved complaint-data snapshot. SpamAssassin uses spam and legitimate email from its public archives. The dataset manifests list the exact labels, source identifiers, selection settings, and exclusions [12]. Each split is balanced across the selected labels. Checks found no duplicate cases or content overlap between the prepared splits or with earlier development material.

Before the reported evaluation, the LEDGAR hybrid was selected using separate development examples and a 100-clause validation check. It was then fixed and evaluated on the 1,000 different clauses reported here as LEDGAR. The earlier selection check is not a second result in this paper. CFPB and SpamAssassin results come from their 100-case validation sets; their reserved tests were not run because the candidates failed the acceptance criteria. This difference in evaluation stage limits direct comparisons across tasks.

3.2 How the baseline and hybrid workflows run

For each case, the baseline sends the input, allowed labels, and English SOP to the model. The scorer compares the returned label with the expected answer. Token use is the model's reported input-token count plus output-token count, summed across cases.

For the hybrid, the benchmark runner first calls the classifier function in `classify.py` with the input text and allowed labels. LEDGAR and CFPB use explicit text-pattern rules that return a label only when the matches identify one label group. SpamAssassin returns the spam label for a single matching rule. If the script returns a permitted label, the runner uses it without a model call. If the script cannot decide, it returns no label, and the runner sends the case to the model using the hybrid SOP's

provides a framework for assessing an allowed loss [13]; it does not supply the numerical margin used here.

3 Experimental Setup

3.1 Evaluation datasets

We use three public sources: LEDGAR contract clauses through the LexGLUE task [14, 15], consumer complaint narratives from the Consumer Financial Protection Bureau (CFPB) [16], and the SpamAssassin public email corpus [17]. Table 1 lists the examples used in the reported comparisons. These balanced subsets support controlled comparisons; their label frequencies do not represent the natural frequency of cases in production.

fallback instructions. This return-to-model behavior is called fallback.

The saved SOP describes the script as `python classify.py`, but the benchmark imports and calls its Python function rather than asking an agent to execute a shell command. The scripts return a fixed confidence value of 0.99 with a match; that value is a routing marker, not a calibrated probability of correctness. The runner checks that the returned label is allowed. It does not independently verify a 99% confidence threshold. Actual accuracy must be measured from labeled cases.

The evaluated hybrid SOPs also changed the wording of the model fallback. The comparison therefore measures the combined effect of adding rules and changing those instructions. In LEDGAR, the fallback became shorter during candidate generation; prompt shortening was not a separately specified architectural change. A separate comparison with unchanged fallback wording would be needed to isolate the effect of routing alone. Section 4.1 reports where the observed token savings occurred without treating that accounting as such an isolated experiment.

3.3 Acceptance criteria and uncertainty

The same four requirements apply to all three datasets [12]:

1. Both the baseline and candidate must achieve at least 80% accuracy.
2. The lower end of the paired 95% interval for the candidate's accuracy change must be no worse than minus two percentage points.
3. Total model tokens must decrease by at least 5%.
4. The lower end of the paired 95% interval for token reduction must be above zero.

All four must pass. The first requirement checks minimum quality. The second checks whether the data support an accuracy loss no larger than the chosen tolerance. The last two require a meaningful observed

token reduction and evidence that the reduction is positive. Like the accuracy thresholds in Section 2.3, the 5% target is a study setting, not an externally established optimum.

We estimate uncertainty with 5,000 paired bootstrap resamples [18]. Each resample selects case identifiers with replacement and includes both workflows' outcomes for each selected case. We recompute the accuracy difference and token reduction, then use the 2.5th and 97.5th percentiles as the interval endpoints. Pairing preserves the fact that the two workflows saw the same inputs. These intervals describe sampling uncertainty within the prepared task, not reliability under arbitrary future data or model changes.

Figure 3 separates candidate revision from evaluation. We report one paired comparison for each dataset: the same examples are processed by the English-only baseline and the fixed hybrid. The three main comparisons comprise six workflow runs. The earlier LEDGAR selection check is documented in Section 3.1 and retained in the archive, but is not included in these result tables or run totals.

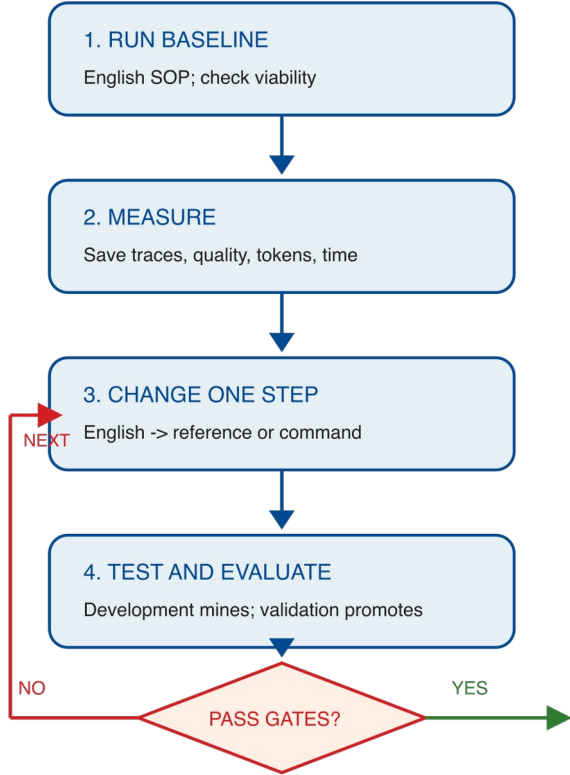


Figure 3. Candidate revision and evaluation use different examples. Development runs guide changes. A fixed candidate is evaluated on validation cases. A passing candidate proceeds to the reserved test without further tuning; a failing candidate is rejected. A revised candidate needs a new, unused evaluation boundary.

3.4 Execution environment and repeated runs

The text experiments used local Ollama with `qwen3:14b` on an Apple M5 Pro with 48 GB unified memory. The main runs were sequential, with temperature set to zero, thinking and streaming disabled, JSON output, and a 128-token response limit. Exact model identifiers and implementation details are recorded in Appendix B rather than in the main description.

We also performed three additional paired runs per dataset using the same fixed SOPs and prepared cases. These runs reused the same LEDGAR, CFPB, and SpamAssassin examples listed in Table 1. They check repeatability; they are not new tests on unseen examples. Each pair used the same inference seed and runtime configuration for its two variants. A seed sets the starting state for a pseudo-random procedure. Dataset selection, model inference, and bootstrap resampling use seeds for different purposes; changing an inference seed does not create a new dataset split.

The repeated runs used a more explicit runtime configuration, including structured output and two parallel requests. Two-worker concurrency was a constraint of the available local execution environment, not a requirement of PLaND or a factor tested by the experiment. Because these settings differ from the original sequential runs, we report the repeated results separately in Appendix A. The three repeats across three datasets add nine pairs, or eighteen workflow runs, for twelve pairs and twenty-four runs in the evidence reported here. This count excludes development, the earlier LEDGAR selection check, and other archived experiments.

4 Results

Table 2 presents the main results. All accuracy and token comparisons show the natural-language baseline first and the hybrid second. A pass means that all four criteria in Section 3.3 were met; it does not mean that accuracy was identical.

Table 2. Main evaluation results

Dataset and stage	Accuracy	Total model tokens	Token reduction	Decision
LEDGAR, 1,000 cases	93.5% → 92.7%	376,088 → 225,573	40.02%	Pass
CFPB validation, 100 cases	79.0% → 72.0%	60,514 → 35,247	41.75%	Reject; test not run
SpamAssassin validation, 100 cases	90.0% → 86.0%	238,077 → 228,359	4.08%	Reject; test not run

Table 3 supplies the uncertainty estimates used for these decisions. Accuracy changes and their intervals are in percentage points. Token-reduction intervals are percentages of baseline token use.

Table 3. Paired uncertainty estimates for the main comparisons

Dataset and stage	Accuracy change	Accuracy change 95% interval	Token reduction 95% interval
LEDGAR	−0.8 points	−1.6 to 0.0 points	37.00% to 43.08%
CFPB validation	−7.0 points	−13.0 to −2.0 points	31.30% to 53.14%
SpamAssassin validation	−4.0 points	−9.0 to +1.0 points	1.02% to 8.50%

4.1 LEDGAR

On the 1,000 LEDGAR clauses, the baseline classified 935 cases correctly and the hybrid classified 927 correctly: eight fewer correct answers, or a 0.8-percentage-point decrease. The paired interval ranged from a 1.6-point decrease to no change. Its lower end remained within the study’s two-point tolerance, so the quality criterion passed. This result supports acceptance under that tolerance, not a claim that quality was unchanged.

The hybrid handled 411 clauses through the script and sent the remaining 589 to the model. It therefore removed 41.1% of model calls and reduced tokens from 376,088 to 225,573, a 40.02% decrease. Of the 411 script-routed clauses, 395 were classified correctly, giving measured routing accuracy of 96.11%. This illustrates why the script’s fixed 0.99 marker should not be read as measured precision.

The token records distinguish two locations of savings. The baseline used 146,366 tokens on the 411 clauses that the hybrid handled without the model. On the remaining clauses, tokens decreased from 229,722 to 225,573, a difference of 4,149. Thus, 97.24% of the total 150,515-token saving occurred on bypassed calls, and 2.76% occurred on the fallback cases. The fallback SOP was shorter, but this accounting does not separate the effect of shorter input instructions from changes in generated output. The combined SOP change remains a limitation of attributing the full result to deterministic routing.

4.2 CFPB

CFPB’s hybrid reduced model calls from 100 to 58 and tokens by 41.75%, but accuracy fell from 79% to 72%. Both variants were below the 80% minimum, and the seven-point observed decrease also failed the relative-quality requirement. The candidate was rejected and the

reserved test was not evaluated. Lower token use did not compensate for the loss in classification quality.

4.3 SpamAssassin

SpamAssassin’s hybrid reduced model calls from 100 to 97. Accuracy fell from 90% to 86%, and token use fell only 4.08%. It failed both the allowed-loss criterion and the 5% token-reduction target, so the reserved test was not evaluated. The three script-routed emails were classified correctly by both variants; the observed accuracy decrease occurred among cases sent to the model. This makes the fallback wording relevant even when the executable rules themselves make no errors on the routed cases.

5 Discussion

5.1 Model reasoning as a resource

The main systems argument is that non-deterministic reasoning should be allocated where it is needed rather than applied uniformly across a workflow. Business processes often contain both stable and ambiguous decisions. When stable regions are repeatedly routed through a language model, the workflow spends model calls and tokens on work that ordinary computation may be able to execute directly.

PLaND provides a controlled way to move that boundary. The English baseline offers a flexible starting point when the structure of the task is not fully known. Development runs reveal errors, repeated reasoning, and expense. The evolver skill guides a host agent to propose a bounded change. Evaluation then determines whether the resulting workflow meets the chosen requirements. This approach may be useful in domains such as fraud detection, where known patterns can be encoded directly while novel narratives or combinations still require

contextual reasoning. Fraud detection itself was not evaluated here.

The useful unit of change is not necessarily an entire workflow step. LEDGAR shows that some inputs to a semantic classification task can be handled by explicit rules, while the remaining inputs still require the model. Contract-clause classification remains a semantic problem overall. Its more recognizable cases nevertheless offer opportunities to avoid model calls, beyond mechanical operations such as counting or schema validation.

The benefit must be considered alongside the error tolerance. LEDGAR saved roughly two fifths of tokens but produced eight additional errors per 1,000 test cases. Whether that tradeoff is acceptable depends on the application. CFPB and SpamAssassin did not satisfy the same evaluation criteria. The three results support evaluating each proposed substitution rather than assuming that fewer model calls imply an acceptable workflow.

5.2 What token savings measure

PLaND's generation instructions ask the agent to consider resource use beyond tokens, including CPU, memory, storage, network access, caching, and setup work. The framework can represent different expense objectives. In the experiments reported here, however, the acceptance decision used model tokens as its only efficiency measure. Model-call counts explain the mechanism; they are not a second independently optimized objective.

Tokens and calls are useful measures of model use, but they do not directly establish dollar, energy, or end-to-end latency savings. Local inference time can depend on model loading, hardware utilization, and concurrency. Code also has execution and maintenance costs. Claims about those costs require corresponding measurements and are outside the present evaluation.

5.3 From classification tasks to business workflows

The retained tasks resemble individual decisions within business processes. They do not include long-running state, user interaction, external side effects, or recovery from partial failure. Extending the result to a complete workflow would require evaluating those behaviors as well as label accuracy. The final test evidence is also limited to LEDGAR, one local model, and a balanced subset of ten labels. The CFPB and SpamAssassin outcomes are validation results, not estimates from their reserved test sets.

Temperature-zero decoding and the small number of repeated runs further limit conclusions about variability. As Appendix A shows, both the baseline and hybrid produced identical labels across the three repeats within each task. This supports repeatability under the tested settings, but does not demonstrate that hybrid execution is less variable than the baseline.

6 Future Work

The next step is to evaluate candidate generation directly. A host agent should receive a recorded set of

development runs, propose code under fixed permissions, and have each candidate assessed on unused examples. Repeating that procedure would measure how often the agent finds a useful rule and how much development work it requires. It would separate the ability to discover an improvement from the ability to execute an already saved improvement.

A second direction is to evaluate complete workflows with state, tool calls, and recoverable failures. The scorer would need to check whether the task was completed correctly, not just whether a label matched. Such experiments would test whether local token savings remain useful when deterministic steps interact with a wider process.

A third direction is to study maintenance over time. Input distributions can change, making a previously acceptable rule unreliable. A longitudinal evaluation could test monitoring, rollback, and returning affected inputs to the model. These mechanisms are proposed future work, not features validated by the present experiments.

Further comparisons should include other models, languages, and methods for generating code from examples or execution records. A routing-only comparison that preserves the entire baseline fallback would clarify the contribution of each change. Application-specific error costs and direct resource measurements would make acceptance decisions more relevant to deployment.

7 Conclusion

PLaND is a methodology for reducing model-mediated work through evaluated changes to an English SOP. Its central mechanism is selective code execution with model fallback. On LEDGAR, the evaluated hybrid reduced model calls by 41.1% and tokens by 40.02%, with accuracy decreasing from 93.5% to 92.7%. It passed the study's stated two-percentage-point tolerance. CFPB and SpamAssassin did not pass validation, and their reserved tests were not evaluated. Three additional paired runs per dataset reproduced those decisions. These results demonstrate the usefulness of testing both quality and token use before accepting a substitution, while leaving automatic rule discovery and complete production workflows for separate evaluation.

Acknowledgements and Disclosures

The authors used AI-assisted tools for coding, experiment support, and manuscript preparation. The authors are responsible for the reported results and the final text. No external funding was received, and the authors declare no competing interests.

Data and Code Availability

The [PLaND repository](#) contains the methodology skills, task runners, saved SOPs, and evaluation records [12]. The reported comparisons use LEDGAR's confirmatory-test files and the confirmatory-validation files for CFPB and SpamAssassin in the [LEDGAR results](#), [CFPB results](#), and [SpamAssassin results](#) folders. Their [variance-study-20260903](#) subfolders contain the repeated runs. Dataset selection

manifests reside alongside each experiment. These links pin the evidence snapshot rather than a moving branch.

The repository retains outputs, comparisons, source identifiers, and hashes. Raw datasets are subject to their source terms. The CFPB inputs were prepared from a saved local API snapshot that is not redistributed in the repository. Consequently, the saved outputs can be audited, but an exact rerun requires that same snapshot; a fresh download from the live database is not an identical replacement. The paper’s three-dataset scope does not

Table 4. Three additional paired runs per dataset

Dataset and reused split	Accuracy in every run	Mean total model tokens	Mean token reduction	Decision in all three runs
LEDGAR, 1,000 cases	93.7% → 92.9%	376,090 → 225,575.33	40.02%	Pass
CFPB validation, 100 cases	78.0% → 71.0%	58,372 → 33,120	43.26%	Reject
SpamAssassin validation, 100 cases	88.0% → 85.0%	188,384 → 178,880.67	5.04%	Reject

Within each dataset and variant, predicted labels were identical across the three runs, so the sample standard deviation of accuracy was zero. Baseline tokens were identical across runs. Hybrid tokens ranged from 225,575 to 225,576 for LEDGAR and from 178,880 to 178,881 for SpamAssassin, with sample standard deviation 0.58 tokens in each case; CFPB used 33,120 tokens in every run. Model calls remained 1,000 to 589 for LEDGAR, 100 to 58 for CFPB, and 100 to 97 for SpamAssassin.

LEDGAR’s paired accuracy intervals were −1.6 to −0.1, −1.6 to 0.0, and −1.6 to 0.0 percentage points, all within the two-point tolerance. CFPB failed both the absolute and relative accuracy requirements in all three runs. SpamAssassin exceeded the 5% token target in these runs but still failed the allowed-loss requirement. Because both variants had zero cross-run label disagreement, the repeats do not show a reduction in output variability attributable to the hybrid.

Appendix B Reproduction details

The repeated-run preflight records Ollama 0.33.0 and qwen3:14b with model digest bdbd181c33f2ed1b31c972991882db3cf4d192569092138a7d29e973cd9debe8. The recorded environment uses an Apple M5 Pro with 48 GB unified memory. The original runner requested JSON output, disabled thinking and streaming, set temperature to zero, and capped generated output at 128 tokens. It did not explicitly set a context-window option.

The repeated-run configuration used a 4,096-token context, the same 128-token output cap, and an exact JSON schema for the label and confidence fields. It preloaded and retained the model, enabled Flash Attention and q8_0 KV cache, kept one model loaded, and allowed two parallel requests. Pair order was baseline then hybrid, hybrid then baseline, and baseline then hybrid. The same inference seed was used within a pair; the three pairs used

remove other historical experiment records from the archive.

Appendix A Repeated runs

The repeatability check introduced in Section 3.4 ran each fixed baseline and hybrid three times on the same prepared cases. Table 4 reports the results separately because the runtime configuration changed. These are repeated measurements of existing evaluation sets, not additional independent test samples.

different seeds. Exact seed values, commands, timestamps, and environment records are preserved with the run manifests rather than repeated in the main text.

The comparison code sorts records by case identifier and uses 5,000 paired resamples. Its token bootstrap uses the configured comparison seed plus one; this is separate from the model’s inference seed. The saved records also include Wilson intervals for individual accuracies and an exact McNemar test, but neither determines acceptance under the four criteria in Section 3.3. Reproduction checks validate file hashes and tests without rerunning model inference or changing a held-out result.

References

- [1] Agent Skills. Specification. [Official documentation](#). Accessed September 5, 2026.
- [2] LangChain. Skills in DeepAgents. [Official documentation](#). Accessed September 5, 2026.
- [3] LangChain. LangGraph overview. [Official documentation](#). Accessed September 5, 2026.
- [4] Khattab O, et al. DSPy: Compiling Declarative Language Model Calls into Self-Improving Pipelines. 2023. [arXiv:2310.03714](#).
- [5] Li Z, et al. AutoFlow: Automated Workflow Generation for Large Language Model Agents. 2024. [arXiv:2407.12821](#).
- [6] Zhang J, et al. AFlow: Automating Agentic Workflow Generation. 2024. [arXiv:2410.10762](#).
- [7] Hu S, Lu C, Clune J. Automated Design of Agentic Systems. 2024. [arXiv:2408.08435](#).
- [8] Yang Y, et al. SkillOpt: Executive Strategy for Self-Evolving Agent Skills. 2026. [arXiv:2605.23904](#).
- [9] Liu Y, et al. SkillRevise: Improving LLM-Authored Agent Skills via Trace-Conditioned Skill Revision. 2026. [arXiv:2606.01139](#).
- [10] Gao Y, et al. SkillReducer: Optimizing LLM Agent Skills for Token Efficiency. 2026. [arXiv:2603.29919](#).

- [11] Kevin C, et al. Evaluating Skills, Not Just Agents: Agentic Continuous Evaluation of Skills. 2026. [arXiv:2608.20614](#).
- [12] Maddipatla Naga Venkata Sai Krishna, Asit Kumar Sahoo. PLaND experiment records and implementation. 2026. [Evidence snapshot f9aa707](#).
- [13] Walker E, Nowacki AS. Understanding Equivalence and Noninferiority Testing. *Journal of General Internal Medicine*. 2011;26(2):192-196. [doi:10.1007/s11606-010-1513-8](#).
- [14] Tuggener D, von Däniken P, Peetz T, Cieliebak M. LEDGAR: A Large-Scale Multi-label Corpus for Text Classification of Legal Provisions in Contracts. *Proceedings of LREC*. 2020:1235-1241. [ACL Anthology](#).
- [15] Chalkidis I, et al. LexGLUE: A Benchmark Dataset for Legal Language Understanding in English. *Proceedings of ACL*. 2022:4310-4330. [ACL Anthology](#).
- [16] Consumer Financial Protection Bureau. Consumer Complaint Database. [Official dataset](#). Accessed September 5, 2026.
- [17] Apache SpamAssassin. Public mail corpus. [Official dataset](#). Accessed September 5, 2026.
- [18] Efron B. Bootstrap Methods: Another Look at the Jackknife. *The Annals of Statistics*. 1979;7(1):1-26. [doi:10.1214/aos/1176344552](#).